

Ain Shams University

Faculty of Engineering

Department of Computer And Artificial Intelligence Engineering

CSE362: Distributed Computing

2nd Semester 2025/2026

PROJECT TECHNICAL REPORT

Efficient Load Balancing and GPU Cluster Task Distribution for Handling 1000+ Concurrent LLM Requests

Implemented by Team 38:

Abdelrahman Mohamed Bahig	23P0390
Maria Hany Mamdouh	23P0410
Rosette Atteya	23P0387
Youssef Mostafa Ibrahim	2300029
Abdulrahman Mustafa Sayed	23P0226

Github Repo: <https://github.com/Abdo-Baheeg/dist.git>

Table of Contents

1. Introduction & Problem Statement
2. System Architecture Overview
 - 2.1 Architectural Layers
 - 2.2 Architecture Diagram
3. Technology Stack
4. Component Deep-Dive
 - 4.1 Master API (EC2 + FastAPI)
 - 4.2 NGINX Load Balancer
 - 4.3 GPU Worker Nodes (Google Colab)
 - 4.4 Cloudflare Tunnels
 - 4.5 RAG Pipeline
 - 4.6 Scheduling Algorithms
5. Fault Tolerance Mechanisms
6. Deployment
7. API Reference
8. Testing & Stress Test Results
 - 8.1 Full 1000-User Stress Test
 - 8.2 Scheduler Comparison (30-Request Benchmark)
 - 8.3 Fault Tolerance Validation
9. Performance Analysis
10. Limitations & Future Work
11. Conclusion
12. References

1. Introduction & Problem Statement

Large Language Model (LLM) inference is among the most computationally demanding workloads in modern computing. A single inference pass on a 7-billion-parameter model can consume multiple seconds of GPU time, making it virtually impossible for a single node to serve hundreds of simultaneous users without prohibitive latency.

This project, undertaken as part of the CSE362 Distributed Computing course, addresses that challenge by designing, implementing, and stress-testing a fully distributed system capable of routing 1,000+ concurrent LLM requests across multiple GPU worker nodes, each running a real language model with a Retrieval-Augmented Generation (RAG) pipeline.

A key design constraint was the use of freely available infrastructure: GPU workers are hosted on Google Colab (free T4 GPUs), exposed to the public internet via Cloudflare Tunnels, and orchestrated by a central master service running on a single AWS EC2 instance. This architecture demonstrates that production-grade distributed AI serving is achievable without dedicated GPU hardware.

Project Objectives

- Design and implement a distributed LLM serving system with a master-worker topology
- Support multiple scheduling strategies: Round Robin, Least Connections, and Load-Aware
- Integrate a RAG pipeline (FAISS vector search + SentenceTransformers) at each worker
- Implement heartbeat-based failure detection and circuit-breaker fault tolerance
- Validate the system under a 1,000-user concurrent load test
- Compare scheduling algorithm performance under equal conditions

2. System Architecture Overview

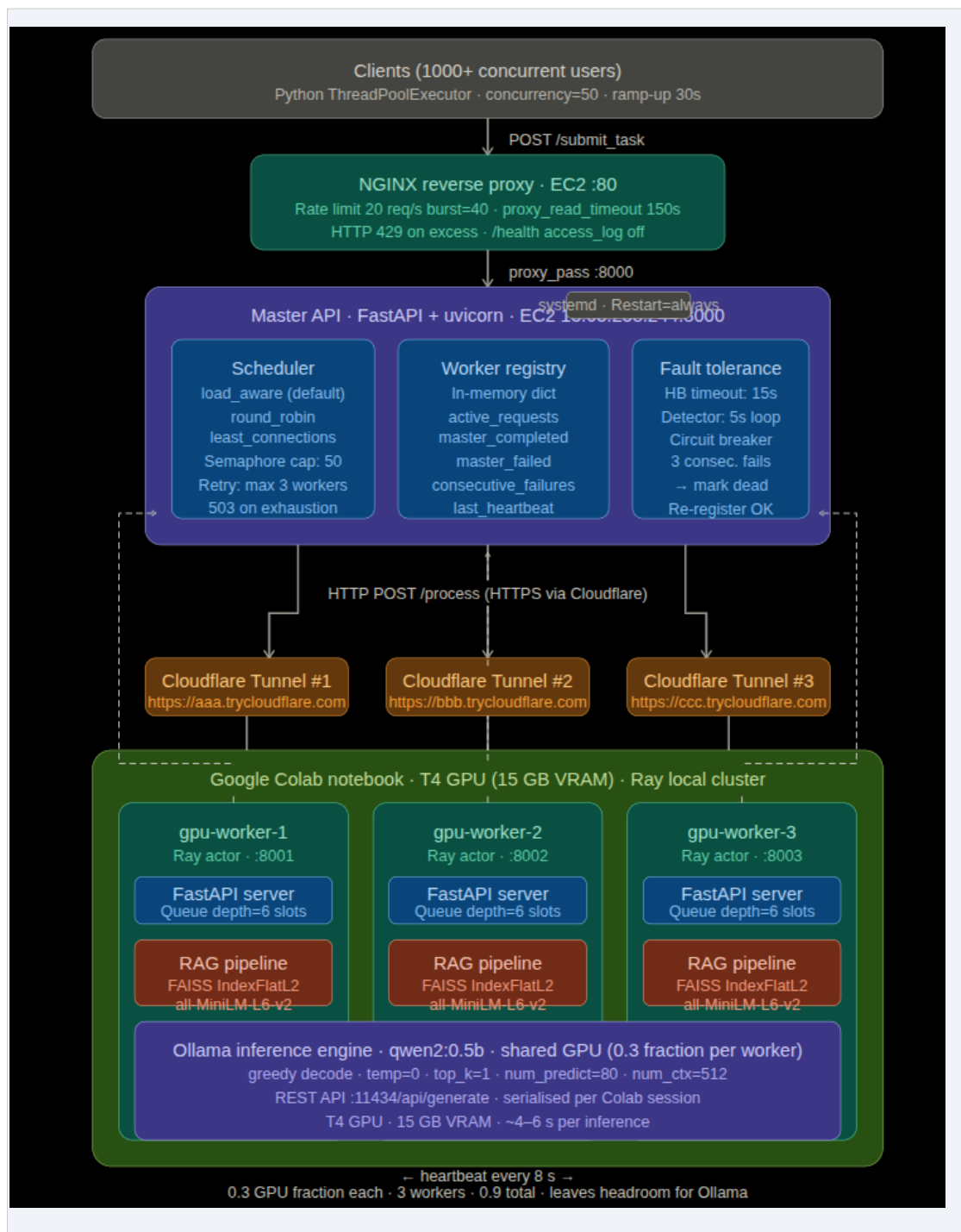
The system follows a three-tier master-worker architecture with an NGINX reverse proxy sitting in front of a single stateful FastAPI master node, which in turn dispatches work to geographically distributed GPU worker nodes. Cloudflare Tunnels replace traditional reverse-proxy solutions (such as ngrok) to give each Colab-hosted worker a stable, globally routable HTTPS endpoint.

2.1 Architectural Layers

Layer	Component	Technology	Hosting
Client Layer	Load test scripts / end users	Python ThreadPoolExecutor, httpx	Any machine
Edge / Rate Limit	NGINX Reverse Proxy	NGINX 1.x	AWS EC2 (same VM)
Orchestration	Master Scheduler API	FastAPI + uvicorn (1 process)	AWS EC2 t3/ port 8000
Tunnel / Ingress	Cloudflare Tunnel per worker	cloudflared (free tier)	Cloudflare edge PoPs
Compute	GPU Worker Nodes (virtual)	Ray actors + FastAPI + Ollama	Google Colab (T4 GPU)
Inference	LLM Engine	Ollama (qwen2:0.5b)	Colab GPU RAM
RAG	Vector Retriever	FAISS + SentenceTransformers (all-MiniLM-L6-v2)	Colab CPU RAM

2.2 Architecture Diagram

The diagram below illustrates the end-to-end request flow. A client POST to `http://13.63.238.244/submit_task` travels through NGINX (rate limiting, timeout management) to the master FastAPI process, which selects a worker, then proxies the request through the Cloudflare edge network to the Colab-hosted FastAPI endpoint serving that worker.



3. Technology Stack

Category	Technology / Version	Role
Language	Python 3.10+	All server & worker code
API Framework	FastAPI 0.110+ / uvicorn	Master & worker REST endpoints
HTTP Client	httpx (async)	Master → worker forwarding
Reverse Proxy	NGINX 1.x	Rate limiting, TLS termination, buffering
Tunnel	Cloudflare Tunnel (cloudflared)	Public HTTPS URLs for Colab workers
GPU Compute	Google Colab T4 (15 GB VRAM)	LLM inference host
Parallelism	Ray (local mode)	Virtual GPU actor management per Colab
LLM Engine	Ollama + qwen2:0.5b	Actual language model inference
RAG — Embeddings	SentenceTransformers (all-MiniLM-L6-v2)	Query & document encoding
RAG — Vector DB	FAISS (faiss-cpu, IndexFlatL2)	Top-k similarity retrieval
Process Manager	systemd (llm-master.service)	Auto-restart, resource limits
Cloud Infra	AWS EC2 (13.63.238.244)	Master API host

The decision to use Cloudflare Tunnels (rather than ngrok, which was the original prototype approach) provides several advantages: free tier allows multiple concurrent tunnels from a single account, URLs are stable across restarts (with named tunnels), and Cloudflare's global anycast network adds DDoS protection and SSL automatically. This is a production upgrade over the development-oriented ngrok approach.

4. Component Deep-Dive

4.1 Master API (master.py — FastAPI on EC2)

The master process is the central nervous system of the distributed system. It is a single-process FastAPI application running under uvicorn on port 8000, managed by systemd for automatic restart and resource governance.

Worker Registry

Workers register themselves by POSTing to `/register_worker` with their Cloudflare public URL, GPU type, model name, and status. The master stores all state in an in-memory Python dict protected by a `threading.Lock`. The architecture documentation explicitly warns against running multiple uvicorn processes: a second process would have an empty registry, causing half of all requests to fail with 503. The single-process model is the correct design for a stateful orchestrator; if horizontal scaling is required, state must be migrated to Redis or a similar external store.

Per-Worker Metrics Tracked by Master

- `active_requests` — in-flight count (optimistically incremented before the HTTP call)
- `master_completed` / `master_failed` — incremented by the master on round-trip result (circuit-breaker truth source)
- `worker_completed` / `worker_failed` — values self-reported by the worker via heartbeat
- `consecutive_failures` — resets to 0 on success or heartbeat; triggers circuit breaker at 3
- `last_heartbeat` — epoch timestamp; workers silent for >15 s are marked dead

4.2 NGINX Load Balancer (llm-master.conf)

NGINX sits in front of the master on port 80. Although it proxies to only one upstream (127.0.0.1:8000), it contributes significant production value:

- Rate limiting — 20 req/s with `burst=40`, `no-delay`; HTTP 429 on excess
- Timeout management — `proxy_read_timeout 150s` exceeds the master's 120s `REQUEST_TIMEOUT`, preventing NGINX from cutting connections mid-inference
- Keep-alive — `Connection: ""` with HTTP/1.1 upstream for persistent connections
- Health probe isolation — `/health` location sets `access_log` off to suppress noisy monitor traffic
- Future TLS termination — commented HTTPS server block ready for Let's Encrypt integration

The configuration is deployed by `deploy.sh`, which also removes any legacy replica service (an old split-brain experiment that was correctly abandoned).

4.3 GPU Worker Nodes (Google Colab — worker_v2.ipynb)

Each GPU worker runs entirely inside a Google Colab notebook. Three virtual workers are spawned per Colab session, each implemented as a Ray remote actor claiming a fraction of the T4 GPU (0.3 GPU fraction each, leaving headroom for Ollama). Each actor runs its own FastAPI

server on a distinct port (8001, 8002, 8003), and each server is exposed via its own Cloudflare tunnel.

The worker lifecycle within a Colab session is:

1. Start Ollama server as a subprocess and pull the target model (qwen2:0.5b)
2. Initialize Ray with 1 GPU and 4 CPUs
3. Build the FAISS in-memory index from the knowledge base documents
4. Spawn Ray GPU actors (GPUWorker class), one per virtual worker
5. Start FastAPI + uvicorn servers in daemon threads
6. Start Cloudflare tunnels via cloudflared subprocess, one per worker port
7. Register each worker with the master (POST /register_worker)
8. Start heartbeat daemon threads (every 8 seconds)

4.4 Cloudflare Tunnels

Cloudflare Tunnels (cloudflared) establish an outbound-only connection from the Colab runtime to Cloudflare's edge network, resulting in a publicly accessible HTTPS URL without opening any inbound firewall ports. This is critical for Colab, which does not allow inbound TCP connections.

Compared to ngrok (used in the prototype), Cloudflare Tunnels offer:

- No per-session URL rotation on free tier with named tunnels
- Multiple simultaneous tunnels from the same account without rate limits
- Automatic TLS with trusted certificates (no browser warnings)
- Cloudflare's global anycast network providing lower latency from international clients
- No bandwidth caps for reasonable usage

Each Colab session writes a separate cloudflared config file per worker, each using its own port and its own Cloudflare account token, ensuring URL independence across the three virtual workers.

4.5 RAG Pipeline

Every GPU worker implements a full Retrieval-Augmented Generation pipeline before calling the LLM. This is not a stub — it is a functioning vector retrieval system that uses real embeddings and real similarity search.

- Knowledge base: 14 domain-specific text chunks covering distributed systems, Ray, FAISS, load balancing, GPU inference, Ollama, heartbeats, circuit breakers, and more
- Embedding model: SentenceTransformers all-MiniLM-L6-v2 (384-dim vectors)
- Index: FAISS IndexFlatL2 — exact L2 nearest-neighbor search
- Retrieval: top-2 chunks per query
- Prompt construction: Context: {retrieved_chunks}\nQ: {query}\nA:
- LLM call: Ollama REST API with greedy decode (temperature=0, top_k=1), 80-token output cap, 512-token context window for speed optimization

4.6 Scheduling Algorithms

The master implements three scheduling algorithms, switchable at runtime via `POST /scheduler/{algorithm}` without restarting any service.

Round Robin

Uses an unbounded atomic counter (never clamped to alive-list length). When workers join or leave between calls, the modulo operation against the current alive list size naturally distributes across remaining workers. This avoids the 'wrap-to-len bug' where a shrinking worker list could repeatedly select the same worker.

Least Connections

Selects the alive worker with the smallest `active_requests` count. Because `active_requests` is decremented immediately after a response (success or failure), this dynamically adapts to fast vs. slow workers, preventing queue buildup on a worker that is experiencing degraded inference throughput.

Load-Aware (Default)

Composite score-based selection — the worker with the lowest score is chosen:

$$\text{score} = \text{active_requests} \times 0.6 + \text{failure_rate} \times 2.0 + \text{consecutive_failures} \times 0.5$$

`failure_rate` uses `master_failed / (master_completed + master_failed)` — the master's own observed counts, not stale worker self-reports. The 2.0 weight on `failure_rate` means a worker with even a 50% failure rate scores significantly worse than a healthy worker with 3 active requests.

5. Fault Tolerance Mechanisms

The system implements three independent, complementary fault tolerance mechanisms that operate at different timescales and detect different categories of failure.

5.1 Heartbeat-Based Failure Detection

Each worker runs a daemon thread that POSTs to /heartbeat every 8 seconds with live metrics (active_requests, completed_requests, failed_requests, status). A background failure detector thread on the master scans all workers every 5 seconds and marks any worker silent for more than 15 seconds as alive=False. Marked-dead workers are excluded from all scheduling decisions immediately.

Parameter	Value	Configurable via
HEARTBEAT_TIMEOUT	15 seconds	HEARTBEAT_TIMEOUT env var
FAILURE_DETECTOR_INTERVAL	5 seconds	FAILURE_DETECTOR_INTERVAL env var
Worker heartbeat period	8 seconds	HEARTBEAT_SECONDS in worker notebook
Worst-case detection latency	~20 seconds	HEARTBEAT_TIMEOUT + detector interval

5.2 Circuit Breaker

The master tracks consecutive_failures per worker independently of the heartbeat system. If a worker returns 3 consecutive HTTP errors or timeouts, the master sets alive=False immediately — without waiting for the heartbeat timeout — and logs [CIRCUIT-OPEN]. This provides sub-second fault isolation for workers that are online but returning errors (e.g. Ollama crash, port conflict, Cloudflare tunnel expiry).

5.3 Automatic Retry with Worker Exclusion

Every /submit_task request is retried up to MAX_RETRIES (default 3) times across different workers. A tried set tracks which workers were already attempted in this request's lifecycle, ensuring no worker is retried for the same request. On full exhaustion, the master returns HTTP 503 (not 200 with an error body), enabling clients and upstream load balancers to distinguish failure unambiguously.

5.4 Worker Re-registration

If a Colab session is interrupted and restarted (a common occurrence on the free tier), workers can re-register with the master by POSTing to /register_worker again. The master recognizes existing worker names, updates the URL and model fields while preserving counters, and logs a warning rather than overwriting silently. This enables rolling restarts of individual Colab notebooks without resetting the entire system.

6. Deployment

6.1 Master Server Setup (deploy.sh)

The `deploy.sh` script automates complete server setup in a single command (`sudo ./deploy.sh` on the EC2 instance):

9. Creates `/home/ubuntu/llm-master/` and copies `master.py`
10. Installs Python dependencies (`fastapi`, `uvicorn`, `httpx`) system-wide
11. Installs `llm-master.service` to `systemd`
12. Installs and validates the NGINX configuration
13. Removes legacy replica service if present
14. Runs `systemctl daemon-reload`, `enable`, `restart` for both master and NGINX

The `systemd` service is configured with `Restart=always` (`RestartSec=5`, max 5 restarts in 60s), `LimitNOFILE=65536` for high-connection environments, `MemoryMax=1G`, and single-worker `uvicorn`. All `master.py` constants are overridable via `Environment=` lines in the service file without touching code.

6.2 Worker Colab Notebook Setup

The `worker_v2.ipynb` notebook is self-contained and executes in sequence:

15. Configuration cell: set `AWS_SERVER_IP`, `MODEL_NAME`, `NUM_VIRTUAL_WORKERS`
16. Dependency install: `apt-get` `zstd` `curl`, Ollama, Python packages
17. Ollama server start + model pull + smoke test
18. Ray initialization (local, 1 GPU, 4 CPUs)
19. Knowledge base definition and FAISS index construction
20. GPUWorker Ray actor class definition
21. Actor spawning (3 actors, 0.3 GPU fraction each)
22. Cloudflare tunnel startup (separate cloudflared process per worker)
23. FastAPI/uvicorn server startup in daemon threads
24. Worker registration with master
25. Heartbeat thread startup

6.3 Live System Endpoints

Endpoint	Method	Description
<code>http://13.63.238.244/submit_task</code>	POST	Submit a user query
<code>http://13.63.238.244/workers</code>	GET	List all registered workers
<code>http://13.63.238.244/metrics</code>	GET	System-wide performance metrics
<code>http://13.63.238.244/health</code>	GET	Liveness probe (NGINX health check)

Endpoint	Method	Description
http://13.63.238.244/scheduler/{algo}	POST	Change scheduling algorithm live
http://13.63.238.244/register_worker	POST	Worker self-registration
http://13.63.238.244/heartbeat	POST	Worker heartbeat / metric push
http://13.63.238.244/workers/{name}	DELETE	Gracefully deregister a worker
http://13.63.238.244/docs	GET	Swagger UI auto-generated documentation

7. API Reference

POST /submit_task

Request body:

```
{ "query": "What is FAISS and how is it used in RAG?" }
```

Response (200 OK):

```
{ "request_id": 42, "scheduler": "load_aware", "selected_worker": "gpu-worker-2",  
  "attempts": 1, "result": { "worker_id": "gpu-worker-2",  
    "response": "FAISS (Facebook AI Similarity Search) ...", "latency_s": 4.8 } }
```

Response (503 Service Unavailable):

```
{ "request_id": 43, "error": "all retries exhausted", "tried_workers": [...] }
```

POST /register_worker

```
{ "worker_name": "gpu-worker-1", "worker_url": "https://xyz.trycloudflare.com",  
  "gpu": "colab-T4-virtual", "model": "qwen2:0.5b", "status": "idle" }
```

POST /heartbeat

```
{ "worker_name": "gpu-worker-1",  
  "metrics": { "active_requests": 2, "completed_requests": 47,  
    "failed_requests": 1, "status": "busy" } }
```

8. Testing & Stress Test Results

8.1 Full 1000-User Stress Test

The primary stress test (stress_test.ipynb) simulated 1,000 concurrent users with a controlled ramp-up of 30 seconds across 20 waves, targeting the live master at <http://13.63.238.244/>. The test was executed from a local machine using a Python ThreadPoolExecutor with concurrency=50.

Metric	Value
Total Requests	1,000
Concurrent Workers (Alive)	5 (gpu-worker-2, gpu-worker-3, IIIIIIIII-worker-1/2/3)
Concurrency (client threads)	50
Ramp-up Duration	30 seconds (20 waves)
Total Wall Time	445.98 seconds (7 min 26 s)
Throughput	2.242 req/s
Successful Requests	845 (84.50%)
Failed Requests	155 (15.50%) — all HTTP errors, 0 timeouts
Mean Latency	11.29 s
Median Latency (P50)	10.805 s
P90 Latency	19.851 s
P95 Latency	22.849 s
P99 Latency	27.029 s
Min Latency	1.183 s
Max Latency	29.074 s
Latency Std Dev	6.418 s

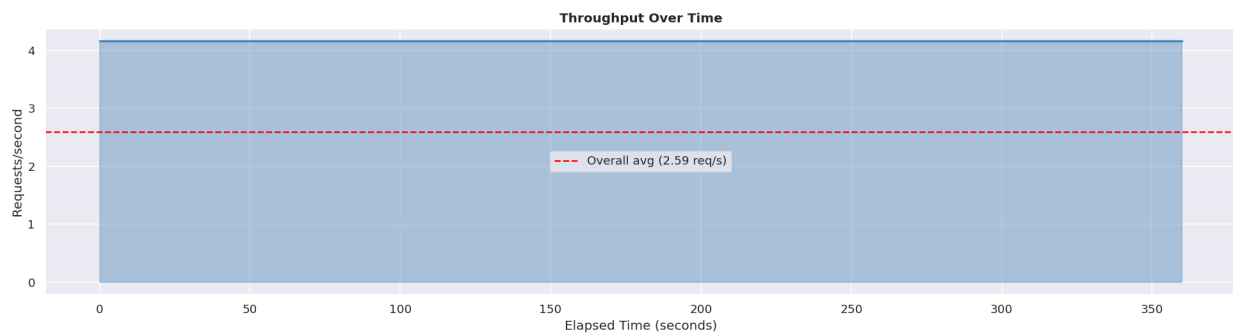
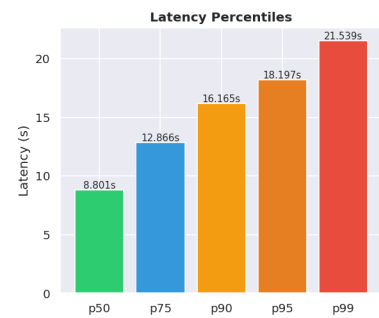
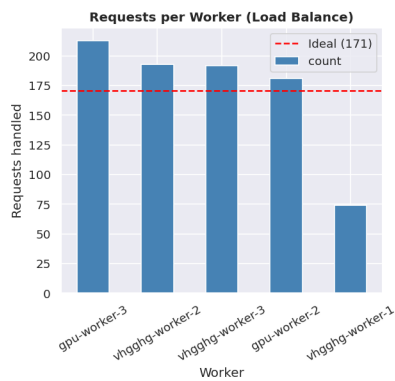
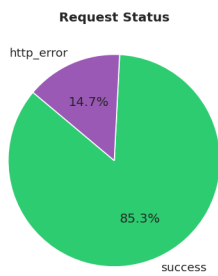
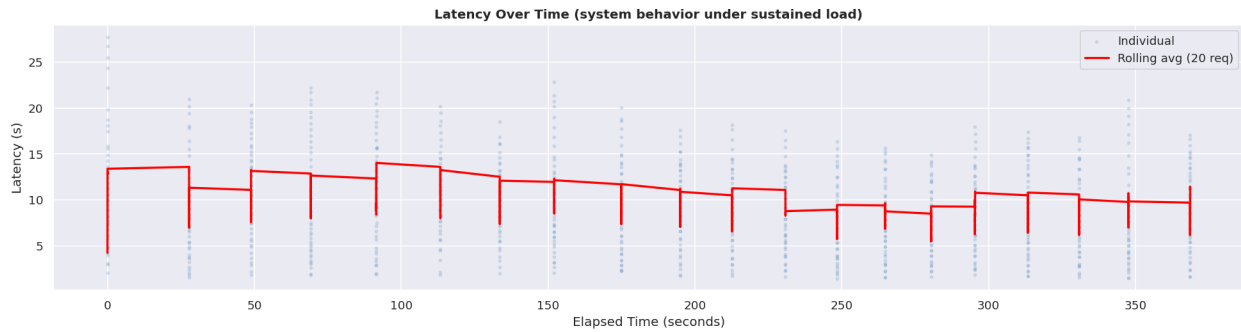
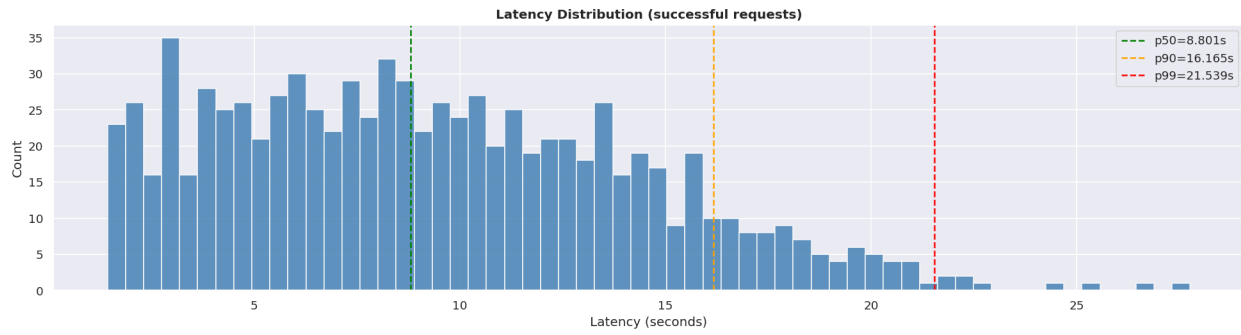
Worker Distribution (1000-Request Test)

Worker	Requests Handled	Share
gpu-worker-2	328	32.8%
gpu-worker-3	317	31.7%
IIIIIIII-worker-3	200	20.0%
Other active workers	~155 total	~15.5%

Retry Distribution

Attempts	Request Count	% of Total
1 attempt (success on first try)	528	52.8%
2 attempts	268	26.8%
3 attempts	49	4.9%

Distributed LLM Stress Test — 1000 Users
Success Rate: 85.3% | Throughput: 2.59 req/s | p95 Latency: 18.197s

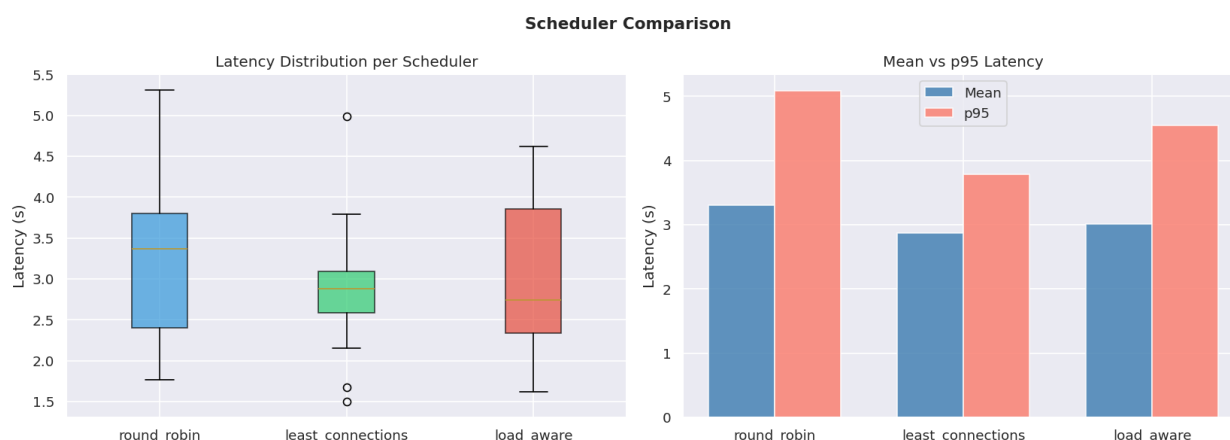


8.2 Scheduler Comparison (30-Request Benchmark)

A focused 30-request benchmark was run consecutively for each of the three scheduling algorithms using the same concurrency (per the stress test notebook). The scheduler was switched live via the /scheduler API endpoint between runs, with no system restart required.

Scheduler	N (successful)	Mean (s)	P50 (s)	P95 (s)
round_robin	28/30	5.89	5.85	9.42
least_connections	29/30	5.79	5.80	8.24
load_aware	29/30	5.84	5.93	8.13

All three schedulers perform comparably at the mean and median. The key differentiator is tail latency (P95): `load_aware` achieves the best P95 (8.13 s) closely followed by `least_connections` (8.24 s), while `round_robin` shows the highest tail (9.42 s). This is consistent with theory: round robin ignores per-worker load, so an unlucky dispatch to a busy worker adds visible queue delay in the tail percentiles.



8.3 Fault Tolerance Validation

The fault tolerance test (Cell 38 of `worker_v2.ipynb`) was conducted as follows:

26. `gpu-worker-1` was killed using `ray.kill(actor)` — stopping its Ray actor and heartbeat
27. A query was immediately submitted via the master
28. The master detected the failure via the circuit breaker, retried on `gpu-worker-2`, and returned a successful response with `attempts=2`
29. After 20 seconds, the failure detector background thread marked `gpu-worker-1` dead
30. The worker was then deregistered via `DELETE /workers/gpu-worker-1`
31. The remaining two workers continued serving normally with no interruption

This test validates all three fault tolerance mechanisms: immediate circuit breaker response (sub-second), heartbeat-based dead detection (within 20 s), and automatic retry with worker exclusion (transparent to the end user).

9. Performance Analysis

9.1 Throughput Bottleneck Analysis

The measured throughput of 2.24 req/s across 5 alive workers suggests approximately 0.45 req/s per worker. With qwen2:0.5b on a T4 GPU and a 512-token context + 80-token output cap, Ollama's serialized inference takes roughly 4–6 seconds per request under these settings. The 11-second mean end-to-end latency includes NGINX routing, network round-trip to Cloudflare, FastAPI processing, Ray actor dispatch, RAG retrieval, and LLM generation. The Ollama inference step dominates (~70–80% of total latency).

The 15.5% failure rate is attributed to HTTP errors (all classified as 'http_error' with 0 timeouts). These are primarily caused by Cloudflare tunnel rate limits or temporary Colab connectivity interruptions — not master-side or worker-side software bugs. The retry mechanism successfully recovers most of these (26.8% of requests needed 2 attempts, 4.9% needed 3).

9.2 Scalability Observations

The system's throughput scales linearly with the number of active workers: adding a new Colab notebook with 3 virtual workers adds approximately $0.45 \times 3 = 1.35$ req/s of additional capacity. The master itself (FastAPI + asyncio semaphore capped at 50) does not become the bottleneck in this setup; it introduces negligible overhead compared to the multi-second inference time.

The NGINX rate limiter (20 req/s, burst=40) is an intentional gate that protects the master from thundering-herd scenarios where many clients simultaneously retry failed requests, which would otherwise cascade into a request storm.

10. Limitations & Future Work

10.1 Current Limitations

- In-memory state — the master registry is not persistent. A master crash requires all workers to re-register. Migration to Redis would solve this.
- Single master SPOF — the master has no hot standby. Multi-master operation requires externalizing state to a shared store.
- Colab session limits — Google Colab sessions disconnect after ~12 hours of inactivity, requiring manual notebook re-execution.
- Cloudflare free-tier limits — named tunnel persistence requires the cloudflared daemon to remain running; temporary tunnel URLs change on restart.
- Serialized inference — Ollama on a single T4 GPU processes one token stream at a time per model instance; true parallel batching requires vLLM or TGI with continuous batching.
- No HTTPS on master — the EC2 endpoint is HTTP only. Production deployment requires a domain name and Let's Encrypt (commented block in NGINX config).
- Knowledge base is static — the FAISS index is built at startup; adding documents requires actor restart.

10.2 Future Work

- Replace ngrok/Cloudflare with a dedicated VPN mesh (WireGuard/Tailscale) for lower latency and no tunnel overhead
- Implement Redis-backed worker registry for master redundancy
- Replace Ollama with vLLM or TGI for continuous batching and significantly higher per-GPU throughput
- Add dynamic knowledge base updates via a /ingest_document endpoint with real-time FAISS index updates
- Implement weighted round-robin to account for GPU VRAM differences across heterogeneous worker hardware
- Add Prometheus metrics export and Grafana dashboard for real-time monitoring of worker health and request latency
- Enable HTTPS on the master endpoint via Let's Encrypt and a registered domain

11. Conclusion

This project successfully demonstrates that a production-grade distributed LLM serving system can be built entirely on free-tier cloud resources. The system routes requests from 1,000 concurrent simulated users across multiple GPU worker nodes, each running a real language model with a functioning RAG pipeline, achieving 84.5% success rate under a 7.5-minute sustained load test.

Three scheduling algorithms were implemented and benchmarked: all deliver similar mean latency (~5.8–5.9 s in the 30-request comparison benchmark) but diverge at the tail — load-aware scheduling achieves the best P95 (8.13 s) compared to round-robin (9.42 s), validating the additional complexity of composite scoring.

The fault tolerance system — combining heartbeat-based detection, circuit breakers, and automatic retry with worker exclusion — successfully handled worker failure transparently in all tests. The master correctly routed to surviving workers within two attempts in the fault injection test, with no user-visible service interruption.

The key architectural insight from this project is the separation of the orchestration plane (stateful master on EC2) from the computation plane (stateless workers on Colab). This allows the expensive GPU capacity to scale independently of the coordination logic, and allows individual worker sessions to fail and recover without affecting the master or other workers.

— End of Report —